

FloatBridge

Keep cash-in and cash-out available

AI DEV FEST 2026 | DIU CPC x upay | Track 05: Merchant & Agent Intelligence

An explainable AI prototype for agent liquidity planning. Prepared 4 October 2026.

Executive summary

FloatBridge helps an operations planner anticipate when mobile-financial-services agents may run short of physical cash or electronic balance. A trained model forecasts hourly cash-in and cash-out demand. A separate planner proposes matched exchanges with a finite synthetic hub. An operator reviews the plan, then the prototype replays the same customer demand to show service outcomes and resource use.

The working browser application includes 12 synthetic agents, four interactive scenarios, adjustable resource constraints, approval, stale-data rejection, event-level accounting, an evidence export and reproducible model evaluation. It has no paid inference dependency and performs no real financial action.

The learned policy produced 12 failed requests out of 5,160 in twelve synthetic evaluation scenarios, compared with 16 for a seasonal forecast using the same planner. The learned policy used four additional visits. This is a limited simulation result, not evidence of production savings or commercial viability.

Problem and user

An agent needs cash to serve a withdrawal and e-float to serve a deposit. A high total balance can still be unusable for the next customer's requested service if its composition is wrong. FloatBridge's primary user is a regional operations planner deciding where a constrained replenishment resource should go.

The contest's Track 05 explicitly lists agent liquidity forecasting. The project links that prediction to an actionable workflow: identify pressure, propose an exchange, obtain approval and measure simulated consequences. The product aims to reduce failed requests without pretending that forecasts guarantee availability.

Implemented product and architecture

User journey

1. Choose an ordinary day, scheduled payday, surprise demand shock or low-demand scenario.
2. Set the exchange cap, visit capacity and delivery delay; inspect agent opening balances and point forecasts.
3. Review proposed cash or e-float exchanges, amounts, arrival times and reasons.
4. Approve or reject the plan. Changing a condition resets approval. A stale balance snapshot stops recommendations.
5. Replay identical requests with and without replenishment, inspect failures and exchanges, and export evidence.
6. Open the separate Evidence lab to compare fixed-threshold, seasonal and learned policies across twelve scenarios.

Architecture

The static interface uses HTML, CSS and JavaScript. Python with NumPy generates data and trains the model before deployment. Exported parameters enable model inference directly in the browser. `engine.js` separates feature calculation, inference, planning, accounting and evaluation. No backend, login or API key is required to demonstrate the product.

Accounting and constraints

A successful cash-in increases agent cash and decreases e-float by the same principal amount. A cash-out reverses those changes. Failed requests do not alter balances. A replenishment is a matched exchange with corresponding changes to the hub's inventory, not newly created money.

The planner respects finite hub cash/e-float, an exchanged-principal cap and a visit cap. Delivery is delayed and rechecked at arrival. Infeasible deliveries are skipped. The heuristic uses projected cumulative imbalance and a reserve target; it does not claim exact optimality or solve geographic routes.

Distinctive contribution

The implementation combines two-balance planning, human review, time-ordered accounting and reproducible outcome comparison. Liquidity prediction itself has prior art. The product's contribution is a transparent, complete decision workflow whose costs and failure cases are visible.

AI approach and data

Synthetic strategy

All identities, demand, balances and operating conditions are synthetic. Twelve agents across ninety days and ten hourly periods yield 10,800 training/evaluation rows. Three agent archetypes, scale factors, time-of-day patterns, weekend effects, known paydays and random noise create varied demand. No real personally identifiable information or upay records are included.

Separate demonstration and policy-evaluation requests are generated at event level. Attempted demand is explicitly retained, including requests rejected by insufficient balances. Production successful-transaction data alone would not reveal all lost demand; obtaining or estimating that information is a future prerequisite.

Learned forecast

Ridge regression predicts log hourly cash-in and cash-out amounts from fourteen time, archetype and scale features. A training-only correction transforms log outputs back to amount estimates. Validation selects regularization from four candidates. This is a trained statistical model with exported coefficients, not a free-form chatbot or a fixed rule masquerading as AI.

The data split is chronological: days 0-59 for training (7,200 rows), 60-74 for validation (1,800), and 75-89 for testing (1,800). Known payday information is available in advance. Unexpected shock realizations are excluded from model inputs. Predictions are point estimates; the emergency prototype does not implement calibrated uncertainty intervals.

Forecast results

Method	Test WAPE	Mean absolute error
Historical seasonal baseline	26.32%	BDT 733.82
Learned ridge forecast	25.80%	BDT 719.31

WAPE is the sum of absolute errors divided by the sum of actual demand. The improvement is modest: approximately 0.52 percentage points. No significance or production-accuracy claim is made.

Decision evaluation and practical impact

Twelve separately seeded scenarios supply 5,160 nonzero attempted requests. All policies receive identical starting agent balances, hub inventory, demand, delivery delay and resource caps. Per scenario, the exchange cap is BDT 60,000, the visit cap is six and delivery occurs after sixty minutes. Actual usage differs.

Policy	Failed requests	Failure rate	Visits	Assumed visit cost
Static threshold	112 / 5,160	2.17%	48	BDT 7,200
Seasonal + same planner	16 / 5,160	0.31%	64	BDT 9,600
Learned forecast + planner	12 / 5,160	0.23%	68	BDT 10,200

The static threshold is a simple 30%-balance reference, not an optimized policy. The seasonal comparator is more informative for AI contribution because it uses the same planning logic. The learned policy avoids four additional failures versus that comparator, at an extra assumed BDT 600 in visit cost. Most of the advantage over the static reference comes from demand-aware planning, not solely from machine learning.

Exchanged principal totals are BDT 415,200 (threshold), BDT 720,000 (seasonal) and BDT 718,800 (learned). Principal exchanged is not revenue or operating cost. Visit cost is assumed at BDT 150; actual provider/agent economics are unknown. A claim of positive ROI would require real costs, customer value and contribution margins.

Group checks and failure cases

Synthetic low-volume agents have 1,287 attempted requests; standard agents have 3,873. Learned-policy failures are 0 and 12 respectively, compared with 0 and 16 for seasonal forecasting. This is a workload-slice check, not a demographic fairness certification. The interface also exposes surprise-shock and delivery-delay scenarios where forecasts or plans may underperform.

Verification

Automated checks cover feature shape, finite/deterministic inference, approval behavior, resource caps, stale balances, accounting conservation, failed-request handling, cash-in/out direction, duplicate delivery, delivery timing and reproducible evaluation. The browser flow was exercised for approval, paused recommendations and evidence navigation. Source code, seeds, trained coefficients and per-scenario evaluation are included for inspection.

Responsible AI, limits and future validation

Responsible behavior

The product displays synthetic-data notices and separates forecasts, rules, actions and results. There is no live money movement, credit approval, real identity data or LLM decision path. Approval is required for the simulated plan, and data freshness can stop recommendations. The ledger checks nonnegative inventories and matched exchanges.

Client-side approval is a demonstration, not production authorization. A production deployment would need authenticated roles, server-side enforcement, audit retention, reliable cash reconciliation and provider-authorized settlement. Publicly exposing actual agent cash balances would be inappropriate; this demo uses fictional balances only.

Limits

Synthetic demand resembles the training process and cannot establish generalization to real customers. The model lacks calibrated uncertainty. The planner uses one batch and a shared delivery delay; it does not optimize routes, repeated dispatch, fees, staffing or external shop cash movements. The simulator does not model customer retries at other agents. The static threshold baseline is not tuned. Counts are small, and actual visit costs differ across policies.

Path toward validation

First confirm the operator workflow and which balance/request feeds actually exist. Next evaluate on appropriately governed historical data, including failed or attempted demand where available. Then run shadow recommendations without execution. Only after those checks should a controlled operator-approved pilot assess service outcomes, costs, acceptance and risk. The organizers describe possible follow-on validation, not promised data access or deployment.

Resources and originality

The project uses NumPy for numerical training and plain JavaScript for runtime inference. AI tools assisted the analysis, implementation, tests and documentation. The team should disclose that assistance and preserve relevant history. No external pretrained model, paid API or production customer dataset is included.

Contest references: General Rules; AI Hackathon Rulebook, especially sections 3-8; Project Guideline & Innovation Playbook, especially Track 05 and sections 10-15. External context: IFC, Digital Financial Services and the Business of Managing Cash; upay's official agent service-location page. These support context, not measured upay impact.

Live deployment: <https://samivai.github.io/floatbridge/>

Public repository and development history: <https://github.com/samivai/floatbridge>

The live approval and replay flow was verified after deployment. The team still needs to provide these links, the report and a recorded video through the official contest submission channel. No official contest submission has been made by publishing this prototype.